

Large Language Model Pruning

Hanjuan Huang ^{a,b}, Hao-Jia Song ^a, Hsing-Kuo Pao ^{a,*}

^a*Dept. of Computer Science and Information Engineering
National Taiwan University of Science and Technology, Taipei, Taiwan 106*

^b*College of Mechanical and Electrical Engineering,
WUYI University, Wuyishan, 354300, China*

Abstract

We surely enjoy the larger the better models for their superior performance in the last couple of years when both the hardware and software support the birth of such extremely huge models. The applied fields include text mining and others. In particular, the success of LLMs on text understanding and text generation draws attention from researchers who have worked on NLP and related areas for years or even decades. On the side, LLMs may suffer from problems like model overfitting, hallucination, and device limitation to name a few. In this work, we suggest a model pruning technique specifically focused on LLMs. The proposed methodology emphasizes the explainability of deep learning models. By having the theoretical foundation, we obtain a trustworthy deep model so that huge models with a massive number of model parameters become not quite necessary. A mutual information-based estimation is adopted to find neurons with redundancy to eliminate. Moreover, an estimator with well-tuned parameters helps to find precise estimation to guide the pruning procedure. At the same time, we also explore the difference between pruning on large-scale models vs. pruning on small-scale models. The choice of pruning criteria is sensitive in small models but not for large-scale models. It is a novel finding through this work. Overall, we demonstrate the superiority of the proposed model to the state-of-the-art models.

Keywords: large language models (LLMs), model pruning, mutual information, trustworthy AI.

1. Introduction

People embrace large-scale models based on large-scale data nowadays. The bigger the models, the better performance we can expect from the models. This is because the big models can have their complexity large enough to cover the high diversity that could be seen from the data. Given this reason, giant industries look for Large Language Models (LLMs) starting from a few years back, and it gets heated since the announcement of ChatGPT at the end of 2022. That shows the evidence that people generally believe the larger the models the more problems can be solved by the models.

The progress of LLMs can be seen by various well-known models such as LLaMA2 [1], PaLM-2 [2], and their followers for various scenarios given a diverse set of data such as LLaMA-Pro-8B [3] and Med-PaLM 2 [4]. At the same time, some others are focused on proposing relatively small models to achieve the same model effectiveness, such as TinyLlama-1.1B [5] and ORCA-2 [6]. Other threads of development includes the Gemini-based structures, such as Gemini 1.5 [7] and Gemma [8]; also, InternLM2 [9] from an independent research group, not to forget the still close-to-the-top-performance one, GPT-4 [10] which top others on various scenarios and receive quite a rich set of comments from the general public.

LLMs are welcome if we do not have storage or computation limitations. Therefore, applications like edge computing cannot be the target when LLMs are among the essential parts of the system. But even if

*Corresponding author

Email addresses: huanghanjuan66@gmail.com (Hanjuan Huang ) , hjh5911@gmail.com (Hao-Jia Song ) , pao@mail.ntust.edu.tw (Hsing-Kuo Pao )

we do not have the limitations, we should keep the storage or computation constrained in as small size as possible for the sack of green concerns. As an alternative viewpoint, we can ask for small models to avoid the possibility of overfitting and hallucination. The smaller the models are, the less likely the models can run into those cases.

In this work, we propose a pruning method that is designed especially for LLMs. The strategy is based on some understanding of the models, such as exploring the direction of model explainability borrowing the Tishby’s deep learning theory [11, 12, 13]. With the understanding of the neurons’ content from the representation layer, we have clues to prune the model where the neurons with redundancy shall be dropped and the neurons with essential information can be kept.

The pruning on large and small-scale models can face very different situations. In large-scale models, neurons are considered with a few possible substitutions. That is, we usually have an over-complete representation in a large-scale model with more than necessary neurons in the representation layer. In this case, two or more neurons could own similar meanings or importance and the pruning on either one can produce resulting models with similar effectiveness. On the other hand, pruning on small-scale models can be crucial in the sense that the number of neurons is close to the minimum required number of neurons and pruning on such models needs careful treatment with as precise computation as possible. In this work, the proposed pruning method, with the help of Mutual Information (MI) estimation can serve the need for such precise computation that leads to effective compressed models.

The most recent progress in LLM compression includes direct network pruning, knowledge distillation, quantization, and low-rank factorization. The different methods enforce the compression on different network parts. Direct network pruning [14, 15, 16, 17, 18] is aimed to remove redundant or unimportant components (neurons or weights) to have a small network to replace a large network to fulfill the goal. The knowledge distillation [19, 20, 21] is focused on finding a small, student model to learn what can be offered from a large, teacher model. Mimicking the prediction ability of the large model from a small model, we achieve the goal of model downsizing. Quantization [22, 23, 24] provides another approach which saves space by utilizing integers or discrete numbers to substitute floating-point numbers in networks. The computation time may also be saved by such a design. The last approach, called low-rank factorization [25, 26], emphasizes replacing the large weight matrix with a small weight matrix to reach the goal of model compression.

To focus on the family of network pruning techniques, we consider two types of methods, the structured and unstructured ones. The overall goal is to remove redundant components from the models. First, the structured pruning simplifies an LLM by removing the entire structural components, such as neurons, channels, or layers while keeping the network structures [27, 28]. On the other hand, the unstructured pruning [18, 29] is aimed at pruning the redundant neurons or links. Given the approach of deleting individual parameters, we may suffer from the irregular sparse structure problems. To compare between the two, the structured pruning can be deployed to various edge devices directly, while the unstructured pruning needs to be accompanied by the assistance of extra software or hardware treatment to complete the task [28].

We have to keep in mind a few items when designing an appropriate compression method for LLMs. First, quite a few LLMs were trained mainly based on unlabeled data. Some of these types of pre-trained LLMs can be used by third-party developers for various downstream tasks. Therefore, we do not know and we cannot assume any label or contextual information provided when building the LLMs. That is to say, we do not know what could be the real mission when building the models or applying any compression to the models [30]. After all, we better assume only an unsupervised type of compression applied to LLMs. Second, most pruning methods need retraining to maintain the models’ performance. This retraining procedure inevitably needs to consume various resources [30] and should be avoided if possible. In LLMs, the Feed-forward Network (FFN) is the major part of the models in terms of the models’ size and it attracts the most computation [31]. Therefore compressing this part of a network means a lot if refers to the saving of space and time complexity. All the above points motivate us to search for an unsupervised LLM compression method that needs no further retraining after the compression is done. Moreover, we prefer one pruning method that can be applied to FFN in particular and that is structured typed.

We utilize an information-based estimation method to measure the information stored in network neurons to decide how we can operate the compression procedure. An MI-based estimator helps to find the relation

between two groups of random variables. Both the linear and nonlinear relations between random variables can be found by such MI estimation [32]. In this work, we estimate the MI value to decide how related between two random variables that correspond to two hidden neurons. When two neurons own a high MI value, they share overlapped information and one of them can be pruned without too much information loss and the overall procedure of model pruning is operated similarly to produce small-scale models in the end.

The aforementioned idea is simple to understand but not trivial to operate in reality due to the target of this work is to focus on the pruning of LLMs. In general, dealing with the pruning of large-scale deep networks is hard or intractable when the model size or the dimensionality in the hidden layers is beyond a certain scale. The MI estimation cannot be scalable easily and we have to rely on an effective MI estimation when facing high-dimensional multivariate data or hidden layers that own large-scale neurons. Given the studies from Wickstrøm et al. [33], Giraldo et al. [34], and Yu et al. [35], researchers aware that the Rényi’s α -order entropy estimator can help to deal with high-dimensional scenarios. Specifically, a matrix-based Rényi’s α -order entropy estimator can be used to estimate Rényi’s entropy in FC layers while a tensor-based Rényi’s α -order entropy estimator can be used to estimate the Rényi’s entropy in the CNN layers. In this work, we adopt the matrix-based Rényi’s α -order entropy estimator to realize the MI computation and proceed with the pruning algorithm to find a small but similar effectiveness model known as the model pruning given the LLMs.

To estimate the MI between hidden neurons as precisely as possible, one can rely on the result offered by Rényi’s α -order entropy estimator. We point out that a parameter in the estimation, the kernel width parameter σ is a key to having precise estimation results. Moreover, in this work, we propose a novel approach to choose an appropriate parameter σ , therefore to have a better Rényi’s entropy estimation, and can accurately prune the redundant neurons to build a small, but effective deep model in the end.

We summarize the proposed work and its contributions as follows.

- The proposed method is considered an unsupervised approach, which needs no label information to decide the pruning strategy. That gives us less burden when moving to large-scale models which may suffer from the labeled data hungry problem.
- The proposed method is a structural pruning method with easy implementation network-wise.
- No need to retrain or fine-tune the compressed model from the proposed method. That implies the pruning strategy can indeed target the neurons that are redundant. On the other hand, if there needs heavy fine-tuning after the compression to have an effective model, the compression is considered not able to pinpoint the redundant neurons or no clear redundant neurons can be found. Such a weak compression only confirms the correctness of the number of neurons but not the content of the neurons before the compression.
- The method has its superiority over other unsupervised pruning methods. The method also shows some competitiveness even if compared to some of the supervised approaches.
- After all, a new kernel width estimation method is introduced to compute the MI between hidden nodes.

The rest of the paper is organized as follows. We elaborate on the background knowledge that is related to model pruning and LLMs in Sec. 2, which is followed by a detailed explanation of the proposed model pruning methodology and the estimation techniques in Sec. 3. To evaluate the proposed method, we have all the results and discussion presented in Sec. 4 and in Sec. 5 we conclude this work.

2. Past Work

Let us discuss the past work that is related to deep model pruning, especially the pruning which can be applied to LLMs. At the same time, we should keep an eye on the most recent progress of LLMs, and understand what pruning techniques can be used for LLM pruning, given the most recent popular model structures to focus for the time being. To speak of the proposed pruning method, we also need to mention

some different types of pruning methods, as well as the MI estimation which is the metric that we use to guide the pruning procedure.

2.1. Structured Pruning for LLMs

Let us go through the past pruning methods that address large-scale models or LLMs and explain how they can achieve the goal. If we consider the structured pruning for LLMs, we separate the methods into the supervised and unsupervised types. In supervised methods, we focus on the relation between hidden neurons and the label, while the mutual relation between different groups of neurons receives more attention in the unsupervised methods. Which type of pruning should be used also depends on the applications.

First, we start with the supervised pruning methods, which are also the majority in the LLM pruning up to now [36, 37, 38, 39, 40, 41]. Among them, Voita et al. [36] proposed a pruning method based on stochastic gates and a differentiable relaxation of L_0 penalty, which can remove the vast majority of heads without seriously affecting the model’s performance. Liu et al. [37] proposed a structured pruning method for efficient BERT inference (EBERT), which can dynamically prune unimportant heads in Multi-Head Self-attention (MHA) and unimportant channels in FFN with the help of the predictor branch. That means that the labeled data are necessary for their operations. Kwon et al. [38] proposed a three-stages pruning framework, which used a Fisher-based mask search algorithm (labeled data are needed) to decide which heads/filters to prune, then rearranged the pruned heads/filters, and at last tuned the mask variables to recover the output signal for each layer. Yang et al. [39] proposed a model pruning toolkit called TextPruner for pre-trained language models. The toolkit includes two pruning methods: one is supervised method, which used the training loss to measure the importance score of neurons; and the other is the self-supervised method, which used the Kullback-Leibler divergence to measure the importance score of neurons. Park et al. [40] proposed a structured pruning algorithm, named Kprune (Knowledge-preserving), which focused on preserving the useful knowledge of the pre-trained model to minimize pruning errors through an iterative pruning process that consisted of knowledge measurement, knowledge-preserving mask search, and knowledge-preserving weight-tuning. Ma et al. [41] introduced an LLM pruning approach referred to as LLM-Pruner. This method employed structural pruning, selectively eliminating non-essential coupled structures guided by gradient information. The aim is to preserve the majority of the LLM’s functionality to the fullest extent possible.

Some of the discussion on the structured and unstructured pruning can be found in the Sec. 1. Other than that, we point out one of the essential procedures, performing retraining or not after pruning, and exploring this viewpoint through various prior work. The work belonging to the retraining-based group [37, 42, 41] has the entire dataset ready for retraining on the compressed model. On the other hand, the retraining-free methods [38, 40, 43] can skip this procedure as the model after pruning remains to be effective. In general, training or retraining on LLMs is known to be resource-intensive and retraining-free approaches should be favored in this sense. An et al. [43] and Park et al. [40] saved the retraining step, but found some alternative ways to enhance the model’s effectiveness. For instance, An et al. [43] added additional bias terms to recover the output feature maps using the baseline values. Park et al. [40] tuned the weights on the pruned model for model improvement. Also, Kwon et al. [38] asked for label information to help with model improvement even if retraining after pruning is not needed.

Overall, we understand that most LLMs have the models trained without the help of label information. The label information for most LLM tasks is indeed hard to acquire. Therefore, we rely on unsupervised pruning more than before. Nova et al. [30] proposed a gradient-free structured pruning framework to integrate two ranking techniques representative ranking and data-driven ranking, without the help of labeled data

2.2. Feature Selection based on Mutual Information

In a sense, we can recognize the values of all network neurons as feature content. Pruning neurons is equivalent to feature selection in this case. In terms of feature selection, we aim to remove irrelevant features [44] from the set to shrink the model size. The irrelevant features imply two meanings: the ones may own redundancy or the ones help nothing in the focused prediction task from the model. We call the

first one the **unsupervised type** and the second one the **supervised type** and further discuss the pruning strategies of the two types.

If through the MI [32] estimation between different neurons, we may understand the neurons' relation and use the clue to prune the possible non-important neurons from the network. As mentioned before, we can utilize MI to measure the relation either between two hidden neurons or between a hidden neuron and the target information. The first is called the **measurement of redundancy** and the second is called the **measurement of relevancy** [44]

Lewis [45] is the first one to propose the MI-based feature selection method, estimating the MI between a potential feature (or a neuron) and the label, we eliminate the neurons that have the MI result fall below a pre-defined threshold. Peng et al. [46] proposed a criterion called minimal-redundancy-maximal-relevance (mRMR) to select useful features, which checked both the MI between candidate features and the label and the MI between different neurons to decide what could be the features to keep. Apparently, we have to pay attention to the trade-off between redundancy and relevance at the same time to make a final decision, which could be case by case. For instance, in a transfer learning type of scenario, we may focus more on the relevance more because the distribution of the target task could be different from what we can focus on the source task. Zhou et al. [44] proposed another approach called CCMI to integrate MI and correlation coefficient to measure how different features are related. Similarly, they also considered both the redundancy and relevance at the same time such as picking up features with high relevance to the label and low redundancy between different features or hidden neurons. Fan et al. [47] applied the MI-based computation to feature or neuron selection then to model pruning. The redundant neurons got eliminated if the neuron set at layer L after the elimination owns the biggest MI value to the neuron set at layer $L + 1$. In the proposed method, we pay attention to redundancy more than relevance. Moreover, we care about the different situations when a small or a large set of neurons is considered for pruning. Other than the aforementioned issues, we utilize an alternative way to estimate the MI, which provides more precise pruning than prior work.

2.3. Mutual Information Estimation

The MI estimation on deep learning networks is hard, if not intractable due to the large-scale of network structures and data size. The classical binning-based estimator [13] considered quantizing neurons' output to estimate the corresponding probability distribution, which leads to at least three problems: (1) an appropriate decision on the bin size to ensure the estimation precision [48, 49], (2) probability distribution estimation needs a large amount of samples [49], (3) difficult computation for certain activation functions such as ReLU. Some other issues include the possible systematic errors [50] happened in the computation procedure. We may utilize some hyperbolic functions (e.g., tanh) to deal with the last issue. Kraskov et al. [51] proposed a k NN distance-based MI estimation called KSG, to deal with a wide range of activation functions. However, this could rely on a wise decision on the number of neighbors. To deal with other issues, Belghazi et al. [52] proposed Mutual Information Neural Estimator (MINE), adopting a different network, using the gradient descent to realize the MI estimation given high-dimensional random variables. Through their framework, both the dimensionality of neurons and the number of samples can be extended linearly for better estimation precision. However, the method is sensitive to the choice of network, and the converging speed is slow for such a network. Wickstrøm et al. [33] improved from the result of Giraldo et al. [34] and Yu et al. [35] and proposed a novel matrix or tensor-based estimation called Rényi α -order entropy estimator, can estimate MI given high-dimensional multivariate data without estimating the probability of random variables that are involved in the MI computation.

The key to the Rényi α -order entropy estimator is focusing on the estimation of the kernel width parameter. Based on that, we have choices between the supervised learning and unsupervised learning approaches. In supervised learning, an optimal criterion [33] is used, while the Scott's rule [53] is considered for the unsupervised learning case. In a nutshell, they aligned the label kernel matrix and a kernel matrix from a pre-specified layer to approximate the kernel width parameter. In Scott's rule [53], one can estimate the kernel width parameter by checking the data size and the dimensionality of the focused hidden layer. The current approach to applying Scott's rule is focused on the estimation of the whole hidden layer [49].

3. Proposed Method

In this section, we introduce the proposed method in detail. The goal of this work is to have an explainable model and prune the model based on such explainability. The key steps to achieve the goal include: finding a way to confirm the explainability of the model, taking advantage of the model’s explainability to prune neurons in the representation layer based on MI computation, and having an effective estimation of MI to produce the final result. Before we go on to elaborate on the details of the proposed method, we introduce the notations that shall be used in this work.

3.1. Notations

In the ℓ -th transformer encoder, we have K_ℓ neurons in the fully-connected layer of FFN, which are denoted by $Z_1, Z_2, \dots, Z_{K_\ell}$ and we use the random variables $Z_1, Z_2, \dots, Z_{K_\ell}$ to describe the value of those neurons (features) in the FC layer. That is, Z refers to a neuron and Z is the random variable to describe the value on neuron Z .

3.2. Framework

Given a pre-trained LLM, we perform a fine-tuning procedure that is designed for a specific task, then apply the proposed pruning to the fine-tuned model to obtain its compressed version. The compressed model is assumed to have similar behavior to its original model. The overall proposed methodology is shown in Fig. 1

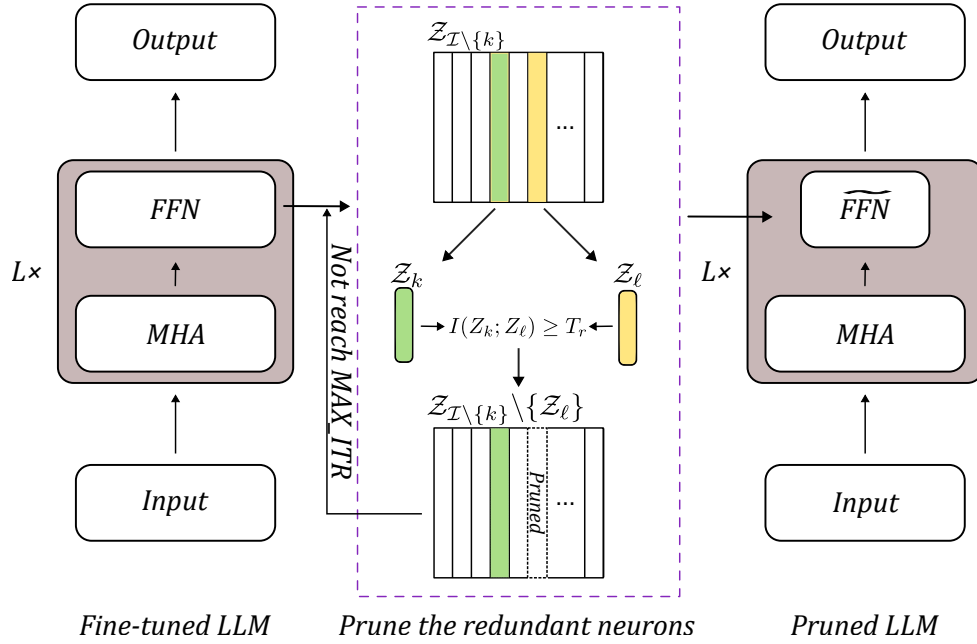


Figure 1: The pruning procedure has the input LLM with L transformer blocks and produces the resulting model with the pruned FFN FC layer. We randomly select two neurons k and ℓ and compute their MI $I(Z_k; Z_\ell)$. If their MI value is smaller than a pre-specified threshold T_r , one of the neurons is deleted. We repeat the step until a maximum number of iterations is reached.

3.3. The Proposed Pruning Methodology

3.3.1. Working on Feed-forward Networks

In one typical design of transformers, we have Multi-Head Self-attention (MHA), followed by one Feed-forward Network in its encoder part. Moreover, each FFN contains two linear transformation layers and a GeLU activation in between.

3.3.2. Redundancy as Feature Selection Criteria

We adopt *mutual information* to measure the relationship between features. Based on the result, we prune features with a certain level of redundancy. In the fully connected layer of FFN, we randomly select two features represented by their corresponding random variables Z_k and Z_ℓ , and compute their mutual information as $I(Z_k; Z_\ell)$. If the values of $I(Z_k; Z_\ell)$ is large enough to show a certain degree of information overlap, we choose one to delete from the feature set.

The whole procedure of the pruning strategy is shown in Algorithm 1.

Algorithm 1 The algorithm of an alternative strategy to select a subset of features that has low mutual information between pairwise features.

Require:

K_r : The no. of remaining features after the alternative pruning strategy

T_r : Maximum allowed feature overlapping

MAX_ITR : The maximum number of iterations

Ensure:

$\mathcal{Z}_{\mathcal{I} \setminus \{k\}}$: The resulting feature set after the pruning strategy

1: $\mathcal{Z}_{\mathcal{I} \setminus \{k\}} \leftarrow \mathcal{Z}$

2: $K_r = K$

3: **for** $i = 1, \dots, MAX_ITR$ **do**

4: Randomly choose two features Z_k and Z_ℓ from $\mathcal{Z}_{\mathcal{I} \setminus \{k\}}$ with their content described by Z_k and Z_ℓ

5: Calculate the mutual information $I(Z_k; Z_\ell)$

6: **if** $I(Z_k; Z_\ell) \geq T_r$ **then**

7: $\mathcal{Z}_{\mathcal{I} \setminus \{k\}} \leftarrow \mathcal{Z}_{\mathcal{I} \setminus \{k\}} \setminus \{Z_\ell\}$

// Turn off one of the similar features

8: **decrease** K_r

9: **end if**

10: **end for**

3.3.3. Clustering Strategy as a Scaling up Option

The pruning algorithm, as shown in Algorithm 1 may not scale well to a large set of neurons or goes quadratically in terms of the number of neurons in its computation. To bypass such a burden, we consider a clustering-based procedure to perform the pruning in a group-based manner. In detail, we cluster features based on their similarity while features with high similarity should go together. Before that, we decide the number of clusters according to different choices of compression rates. When the clustering result is confirmed, we choose one feature, which could be the one closest to each cluster centroid to be the one to keep while all the other neurons of the same cluster should be eliminated after the procedure.

The mutual information is used to decide a metric for the clustering procedure. Given the pairwise distances, we utilize the multidimensional scaling (MDS) [54] to find coordinates in a pre-specified dimensionality. In detail, given two features Z_k and Z_ℓ we compute their mutual information $I(Z_k; Z_\ell)$ and a set of pairwise mutual information is transformed into pairwise distances. Afterward, Eq. 1 helps to find coordinates given a pre-specified space of certain dimensionality.

$$d(Z_k, Z_\ell) = A \exp(-I(Z_k; Z_\ell)), \quad (1)$$

where A is a constant in case if we prefer a distance between 0 and 1. In the formula, the larger the mutual information between Z_k and Z_ℓ , the smaller the value $d(Z_k, Z_\ell)$ is. In the MDS-projected space, two close-by features Z_k and Z_ℓ implies that they share larger mutual information $I(Z_k; Z_\ell)$. Moreover, two features with their large mutual information $I(Z_k; Z_\ell) \geq T_r$ may end up in the same cluster and the pruning strategy in Algorithm 1 could suggest the removal of one of the features in the pair. That is, we group features into a cluster if they own large mutual information. After that, only one feature per cluster is used to be the representative once we obtain the grouping result. In the end, we have features in different clusters if the features have their pairwise mutual information smaller than a threshold T_r .

3.3.4. Subsidiary Condition

The procedure may not produce a unique compression model because the solution to MDS and the selection of representatives may not always be the same. We may suggest some side condition to encourage a decent compression result by trying M random seeds and choosing the best one out of them by the following criteria.

The subsidiary condition is to minimize the difference between the original and the compressed models. We use the Kullback-Leibler (KL) divergence to measure the difference between the original and the compressed model by Eq. 2 if focusing on the representation of both models. Given the original model, we aim to find a compressed model $\mathcal{M}$ that is closet to the original model in its representation distribution if measured by KL divergence.

$$\mathcal{M}_{\text{comp}}^* = \underset{\mathcal{M}}{\operatorname{argmin}} D_{\text{KL}}(p(z_o) || p(z_{\mathcal{M}})), \quad (2)$$

where $D_{\text{KL}}(p_1 || p_2)$ measures the KL divergence between two distributions p_1 and p_2 , z_o and $z_{\mathcal{M}}$ denote the representation of the original and the compressed model $\mathcal{M}$, respectively; $p(z_o)$ and $p(z_{\mathcal{M}})$ are the distributions of z_o and $z_{\mathcal{M}}$, respectively. A small D_{KL} indicates a closer relationship between $p(z_{\mathcal{M}})$, the representation for the compressed model and $p(z_o)$, the representation of the original model.

3.4. Estimate Method of Kernel Width Parameter of Hidden Neuron

To estimate the mutual information between Z_k and Z_ℓ , we compute

$$I(Z_k; Z_\ell) = S_\alpha(A) + S_\alpha(B) - S_\alpha(A, B), \quad (3)$$

where A and B are Gram matrices of Z_k and Z_ℓ , respectively. $S_\alpha(A)$ and $S_\alpha(B)$ denote the matrix-based Rényi's α -order *entropy*, respectively. $S_\alpha(A, B)$ denotes the *joint entropy* of Z_k and Z_ℓ .

The estimation is analogous to the estimation of the mutual information in a reproducing kernel Hilbert space (RKHS), that is, solving the eigenvalues of the kernel matrix to estimate the entropy, and solve the eigenvalues of the Hadamard (entry-wise) product of two kernel matrices to estimate the joint entropy. In this computation, one of the key steps is to choose an appropriate kernel width parameter σ to have a decent estimation.

To have it focus on a single neuron-related calculation, we have to adjust the estimation by integrating the optimal criterion and Scott's rule to estimate the kernel width parameter if one hidden neuron is focused. The process is illustrated as follows.

First, We apply Scott's rule to estimate the kernel width parameter σ^ℓ of the focused hidden layer, we have

$$\sigma^\ell = \gamma N^{\frac{-1}{(4+d)}}, \quad (4)$$

where N denotes the number of samples, d denotes the the number of hidden neurons, and γ is an empirically determined constant. Second, we adopt Eq. 5 to calculate the kernel matrix K_{σ^ℓ} of hidden layer. The RBF kernel is written as:

$$K_\sigma(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_F^2}{2\sigma^2}\right), \quad (5)$$

where $\mathbf{x}_i, \mathbf{x}_j$ denote data points, $\|\cdot\|_F$ denotes the Frobenius norm, σ is the kernel width parameter. Third, we align kernel matrix K_{σ^ℓ} with kernel matrix K_{σ^n} (kernel matrix of hidden layer) by maximizing the kernel alignment loss between these two kernel matrices, the kernel alignment loss [55] is written as:

$$A(K_{\sigma^\ell}, K_{\sigma^n}) = \frac{\langle K_{\sigma^\ell}, K_{\sigma^n} \rangle_F}{\|K_{\sigma^\ell}\|_F \|K_{\sigma^n}\|_F}, \quad (6)$$

where $\|\cdot\|_F$ and $\langle \cdot, \cdot \rangle_F$ denote the Frobenius norm and inner product, respectively.

Thus, we choose our optimal σ^n as Eq. 7:

$$\sigma^{n*} = \arg \max_{\sigma^n} A(K_{\sigma^\ell}, K_{\sigma^n}). \quad (7)$$

Table 1: Data statistics of GLUE datasets. Among them, except for STS-B, which is a regression task, the others are classification tasks.

Tasks	Datasets	Training	Validation	Test	Metrics
Single-sentence	SST-2	67350	873	1821	Accuracy
Inference	QNLI	104743	5463	5461	Accuracy
Similarity and paraphrase	STS-B	5749	1379	1377	Pearson correlation (r) Spearman correlation (r_s)
	MRPC	3668	408	1725	F1 / Accuracy
	QQP	363870	40431	390965	F1 / Accuracy

The best choice of σ^n according to different size of mini-batches, we consider the approach from Wickstrøm et al. [33] to compute the exponential moving average to decide the final value, such as

$$\sigma_{n,t} = \beta \sigma_{n,t-1} + (1 - \beta) \sigma_{n,t}^*, \quad (8)$$

where $\beta \in [0, 1]$ and $\sigma_{n,1} = \sigma_{n,1}^*$. Finally, we obtain the kernel width parameter of every hidden neuron in the hidden layer.

4. Experiment Result

We conduct a few series of experiments to evaluate the effectiveness of the proposed model. The first goal is to understand how accurate the mutual information estimation on the focused hidden neurons is. After that, we need to confirm the effectiveness of the proposed pruning method. Relative FLOPs are used to indicate the compression level of the model, the formula is expressed as:

$$\text{Relative FLOPs} = \frac{\text{FLOPs of pruned model}}{\text{FLOPs of original model}}.$$

The smaller the value of Relative FLOPs, the higher the compression level of the model. We follow the widely adopted definition of Relative FLOPs [56, 38].

4.1. Experimental Settings

We evaluate the effectiveness of the proposed methods using the BERT-tiny model [57] on the General Language Understanding Evaluation (GLUE) [58] benchmark. The BERT-tiny is a (pretraining + fine-tuning) model from [57], which consists of one Embedding layer and two transformer encoder blocks, with a hidden size of 512 for the FC layer in FFN. The GLUE benchmark contains a collection of NLU tasks and we fine-tuned it on five downstream tasks: Single-Sentence Task (SST-2 [59]), Similarity and Paraphrase Tasks (STS-B [60], MRPC [61], and QQP [61]), and Inference Task (QNLI [61]). The batch size was set to 8, 8, 8, 16, and 16 for these tasks, respectively. Additionally, the learning rate was set to $5 \cdot 10^{-4}$, $5 \cdot 10^{-4}$, $3 \cdot 10^{-4}$, $3 \cdot 10^{-4}$, and $3 \cdot 10^{-4}$ for these tasks, respectively. Throughout all experiments, we trained the model using the AdamW optimizer [62] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and conducted a total of 4 fine-tuning epochs. The overall data statistics and corresponding evaluation metrics are shown in Table. 1.

In the estimation of mutual information settings, we random sample 1% of the number of samples per task training dataset to calculate the mutual information, so we can know that the N in Eq. 4 is equal to the random sample number. At the same time, in Eq. 3, we set $\alpha = 1.01$, in Eq. 4, we set $\gamma = 1$, $n = 512$. The batch size of the calculation process is set to 100, which is the same as [48]. Due to the randomness of MDS, we sample 500 random seeds and choose the best one based on Eq. 2.

Table 2: Results of different methods when the relative FLOPs equal to 40%. Abbreviations S, Self-S, and U denote the Supervised method, the Self-supervised method, and the Unsupervised method, respectively.

Methods	S/U /Self-S	Relative FLOPs	SST-2 Acc	STS-B r/r_s	MRPC Acc/F1	QQP Acc/F1	QNLI Acc
BERT-tiny (Original)		100%	83.2	74.3/73.6	81.1/71.1	62.2/83.4	81.5
TextPruner [39]	S	40%	80.8	72.9/70.5	81.3/70.7	62.7/85.3	78.7
Mask-tuning [38]	S	40%	81.7	73.7/70.9	80.7/69.6	61.8/85.3	65.0
Kprune [40]	S	40%	83.1	74.4/72.3	81.0/70.1	61.8/84.2	77.5
TextPruner [39]	Self-S	40%	81.8	70.3/68.7	80.8/70.0	62.8/84.9	76.2
Random	U	40%	80.7	71.0/69.4	80.8/68.7	59.1/84.4	67.2
Weight-Magnitude [16]	U	40%	81.8	71.4/69.6	80.8/68.5	61.2/83.9	67.7
KCM [30]	U	40%	78.8	72.6/70.3	81.1/69.8	61.9/84.0	74.5
Proposed method	U	40%	82.6	72.1/69.2	80.9/69.4	61.2/84.3	77.2

¹ TextPruner [39] includes two methods: one is self-supervised learning, and the other is supervised learning.

4.2. The Results of Model Pruning

In this part, we compare the proposed method to some supervised learning approaches [39, 38, 40], self-supervised learning method [39], and unsupervised learning methods [16, 30], overall three types of comparisons. The proposed method is similar to the weight-magnitude approach [16] and KCM [30], in the sense that all need no labeled data in the pruning procedure. Moreover, the proposed method is a retraining-free approach, which follows the convention from [38, 30, 40]. It is different from the approach adopted on [38, 30, 40] where weight-tuning on the left-out (unpruned) neurons is necessary to confirm the more-than-acceptable network effectiveness. Note that the random strategy has its output as an average of ten trials to reveal the general behavior of the strategy. In all experiments, we fix the pruned percentage to be 1%.

In Table. 2, we show that the proposed model can maintain the performance of the original model for various tasks given the relative FLOPs are set to be 40%. Let us compare the proposed method to the Weight-Magnitude [16] and KCM [30] approaches, both belonged to the category of unsupervised methods, and own some similarities to the proposed method. The proposed method performs superior to the Weight-Magnitude method on the tasks SST-2 and the QNLI while slightly worse on the other three tasks. On the other hand, we also compare the proposed method to another self-supervised method like Yang et al. [39]. The proposed method is superior to other methods on all but the MRPC and QQP tasks. To compare to the category of supervised method, we evaluate the superiority of the proposed method to other supervised methods. In this category, the proposed method performs better than the other two methods on the SST-2 and QNLI tasks. For instance, the proposed method is better than Mask-tuning [38] on the SST-2 and QNLI tasks, and better than Yang et al. [39] on the SST-2 task. For other tasks, the proposed method and the two supervised methods perform similarly.

To clearly show the model performance under different compression rates, we consider the following set of comparisons given five tasks. As shown in Fig. 2, we demonstrate the result for every 1% change of the compression rate. For all the tasks other than STS-B, the proposed method performs better than the random strategy. If compared to the unsupervised learning method proposed by Li et al. [16], the proposed pruning method shows better performance on all except the QQP dataset, and both perform similarly on the QQP dataset. Compared to another method from the unsupervised group, we consider KCM [30]. In this case, both perform similarly on all four except the QQP dataset. The performance on SST-2 is slightly better if by the proposed method.

When checking the self-supervised method such as Yang et al. [39], the proposed method shows its

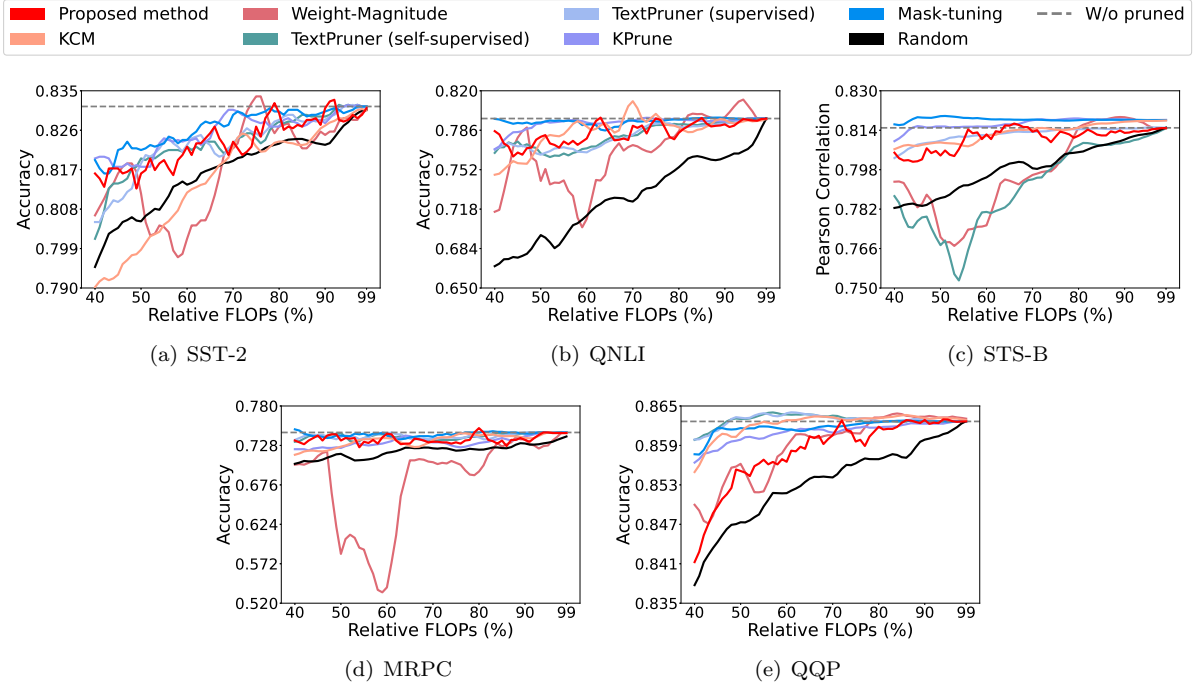


Figure 2: Performance of the proposed method against other methods on BERT-tiny. Testing was conducted using the dev set to control relative FLOPs incrementally by percentage.

advantage on both the QNLI and STS-B datasets, but not on the SST-2 dataset. In the category of supervised learning, the proposed method shows similar results on both the STS-B and QQP datasets given low compression rates, while the proposed method could perform poorly given the high compression rates. On the SST-2, QNLI, and MRPC datasets, all behave similarly.

4.3. Mutual Information between Hidden Neurons Estimation

In this part of experiment, we take a close look about the MI estimation between hidden neurons. We compare the adopted approach and the estimation based on the kernel width parameter tuning (Scott’s rule). Of course, no true value can be obtained to confirm how accurate the MI estimation is, we provide the study on the model test accuracy to provide another support for the pruning effectiveness.

As shown in FIG. 3, the proposed method offers better performance in terms of prediction accuracy than other methods in most cases. We have a clear advantage on the QQP dataset, across all compression rates. Working on the QNLI dataset, the proposed method still keeps an advantage in most cases. On the SST-2 and MRPC datasets, both (the proposed method and Scott’s rule) show similar results. At the last, the proposed method may perform poorly on the STS-B dataset, for at least the low compression rate cases.

4.4. Ablation Study

We take turns studying some alternative approaches to see the possibilities of further improvement from the proposed method.

4.4.1. Mutual Information vs. Pearson Correlation Coefficient

First, we consider an alternative measure to describe how two variables are related to each other, when we need to measure the relation between two groups of neurons. For instance, we may choose Pearson correlation coefficient (PCC) to substitute the role of MI in Algorithm 1 to guide the pruning procedure. The result from both, the MI-based and the PCC-based ones are shown in FIG. 4.

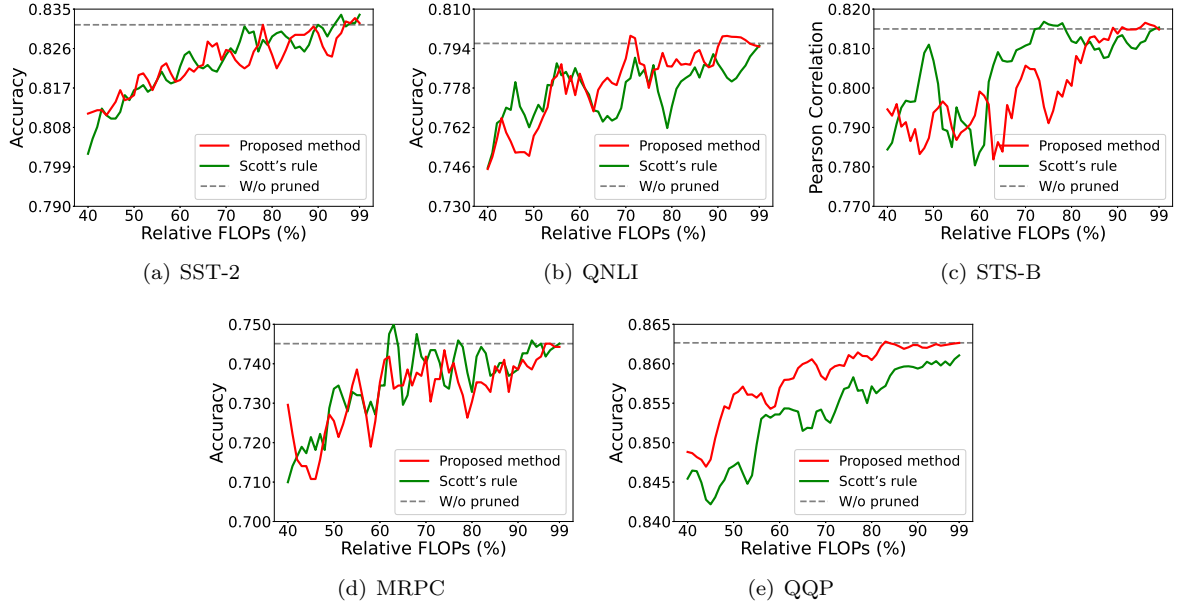


Figure 3: Performance of the proposed method against other MI estimators on BERT-tiny. Testing was conducted using the dev set to control relative FLOPs incrementally by percentage.

In FIG. 4, using MI rather than PCC to describe the relation between neurons performs the best given the high-compression (small remaining model) situations, for all but the STS-B dataset. It implies that MI can offer complex descriptions between different random variables, compared to the one from PCC which covers only simple or linear relationships between random variables. That is to say, the pruning on a large model may not need careful treatment because neurons may find their substitutes easily, while the pruning on a small model needs precise calculations to be done, which could be offered from the MI-based approach.

4.4.2. Data Samples for Mutual Information Estimation

This series of experiments is devoted to studying how large data is enough to estimate the MI value. If a small dataset offers a result similar to the one from a large dataset, then we prefer using a small dataset to save the running time in model pruning.

When estimating the MI value, we rely on the values of the hidden neurons. That is, once the network is built after a converged training, we can sample only a small portion of the input data, and use them to activate the hidden neurons for the estimation. According to our study, the estimation needs no large-scale data to confirm the pruning effect.

As shown in Fig. 5, we take turns to consider 1%, 10%, 50%, or 100% (all) of the complete data to see their pruning performance. We feel lucky to see that different data portions may still provide similar model performance from the pruned model, on all but the STS-B dataset. Note that we need a regression task on the STS-B dataset, while classification tasks are needed on all other datasets.

4.4.3. Different α for Matrix-based Rényi's α -order Entropy Estimation

In this series of experiments, we would like to find out what could be the best choice of α , for the estimation of Rényi's α -order Entropy. As shown in FIG. 6, choosing $\alpha = 1.01$ gives the best result overall for five different tasks.

4.4.4. Sample Number for MDS

The proposed procedure may not always produce the same unique compressed model because the solution to MDS and the selection of representatives may not remain the same. To deal with that, we sample a few

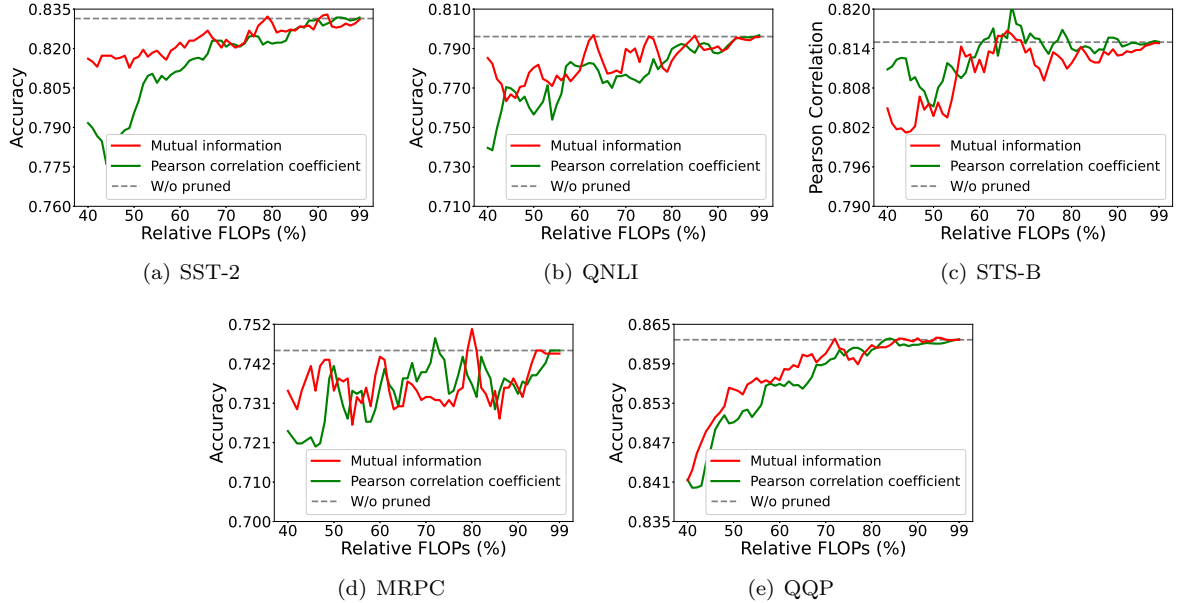


Figure 4: The result based on mutual information or Pearson correlation coefficient computation

random seeds and choose the best one based on Eq. 2. The result from comparing different numbers of samples is shown in Fig. 7.

5. Conclusion

In this work, we proposed a novel method for large-scale language model pruning. The pruning method is based on a precise estimation of the mutual information on the representation neurons. Moreover, a matrix-based Rényi’s α -order entropy is adopted to realize such estimation. In the estimation, we proposed a strategy to find out the kernel width parameter, more effective than previous approaches. Overall, the pruning method needs no label information nor retraining to complete the task. Through a few series of experiments, we can conclude that the proposed method indeed produced the pruned model, more effective, but smaller than those from other state-of-the-art pruning approaches. The superiority covers almost all unsupervised approaches and a few supervised approaches. The result is also similar to that offered by self-supervised learning. In the future, we would like to take an even larger model than what we can process now to confirm the scalability of the proposed method.

References

- [1] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.
- [3] Chengyue Wu, Yukang Gan, Yixiao Ge, Zeyu Lu, Jiahao Wang, Ye Feng, Ping Luo, and Ying Shan. Llama pro: Progressive llama with block expansion. *arXiv preprint arXiv:2401.02415*, 2024.
- [4] Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Le Hou, Kevin Clark, Stephen Pfohl, Heather Cole-Lewis, Darlene Neal, et al. Towards expert-level medical question answering with large language models. *arXiv preprint arXiv:2305.09617*, 2023.
- [5] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

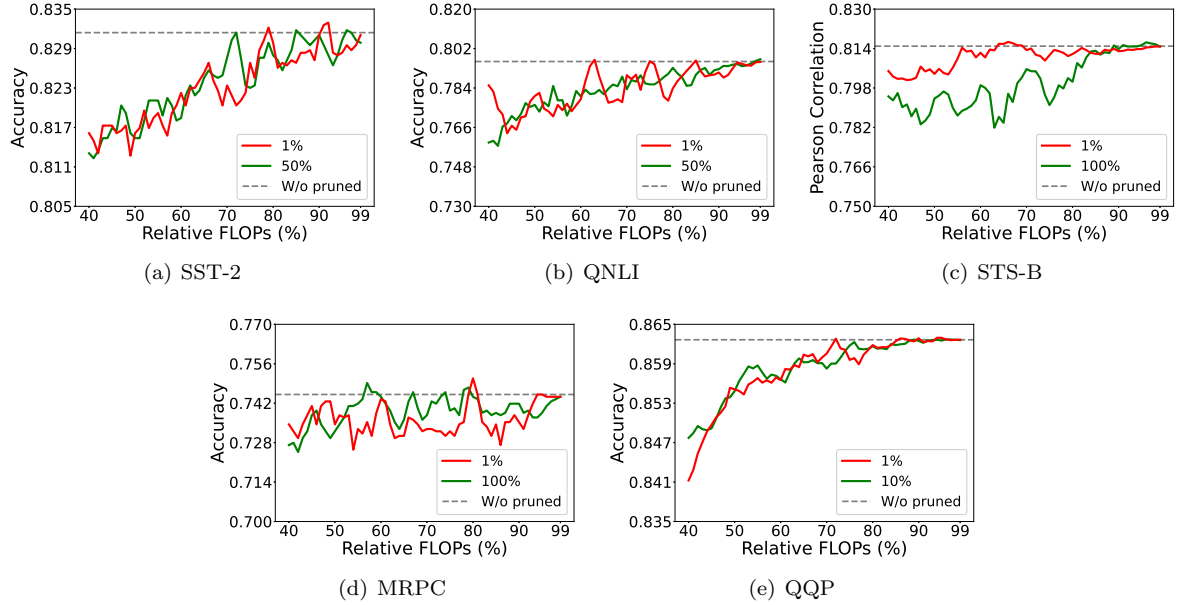


Figure 5: Insignificant difference between the result from either the complete or partial dataset.

- [6] Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Coda, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, et al. Orca 2: Teaching small language models how to reason. *arXiv preprint arXiv:2311.11045*, 2023.
- [7] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [8] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Riviere, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [9] Zheng Cai, Maosong Cao, and Haojiong Chen et al. Internlm2 technical report. *arXiv:2403.17297*, 2024.
- [10] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [11] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [12] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE Information Theory Workshop (itw)*, pages 1–5. IEEE, 2015.
- [13] Ravid Shwartz-Ziv and Naftali Tishby. Opening the black box of deep neural networks via information. *arXiv preprint arXiv:1703.00810*, 2017.
- [14] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. *Advances in Neural Information Processing Systems*, 2, 1989.
- [15] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in Neural Information Processing Systems*, 28, 2015.
- [16] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. *arXiv preprint arXiv:1608.08710*, 2016.
- [17] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 36:21702–21720, 2023.
- [18] Elias Frantar and Dan Alistarh. Massive language models can be accurately pruned in one-shot. *arXiv preprint arXiv:2301.00774*, 2023.
- [19] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [20] Frederick Tung and Greg Mori. Similarity-preserving knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1365–1374, 2019.
- [21] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023.
- [22] Zhenhua Liu, Yunhe Wang, Kai Han, Wei Zhang, Siwei Ma, and Wen Gao. Post-training quantization for vision transformer. *Advances in Neural Information Processing Systems*, 34:28092–28103, 2021.

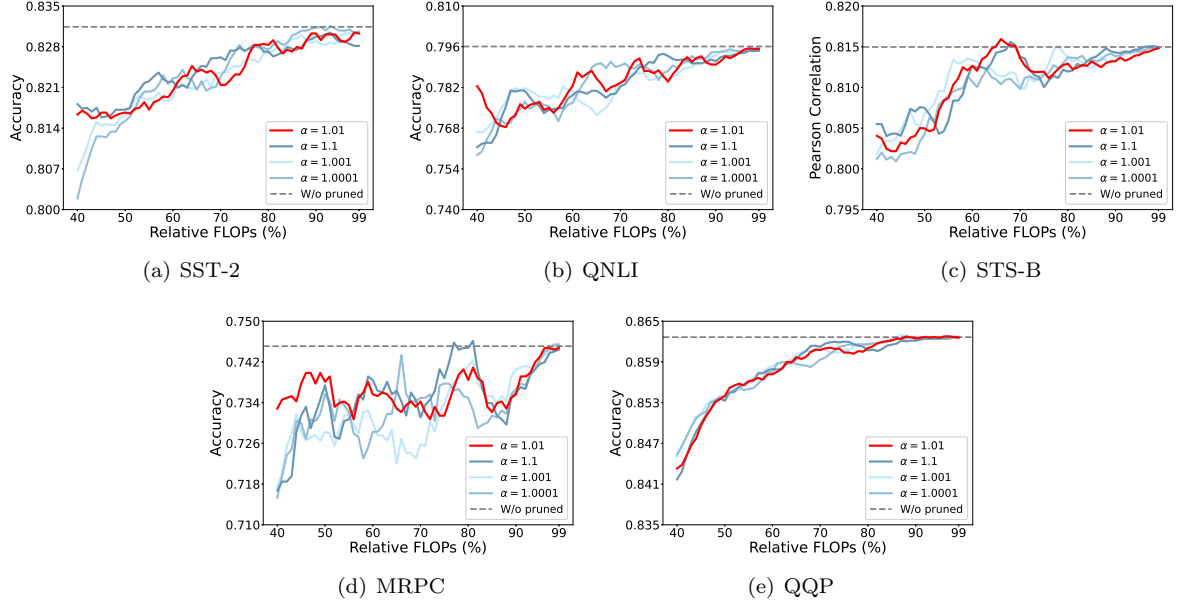


Figure 6: The value of $\alpha = 1.01$ demonstrates exceptional performance in highly pruned models compared to other α values utilized in Rényi entropy estimation.

- [23] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*, 2023.
- [24] Shaojin Ding, Phoenix Meadowlark, Yanzhang He, Lukasz Lew, Shivani Agrawal, and Oleg Rybakov. 4-bit conformer with native quantization aware training for speech recognition. *arXiv preprint arXiv:2203.15952*, 2022.
- [25] Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, and Sanjeev Khudanpur. Semi-orthogonal low-rank matrix factorization for deep neural networks. In *Interspeech*, pages 3743–3747, 2018.
- [26] Xiaoxia Wu, Zhewei Yao, and Yuxiong He. Zeroquant-fp: A leap forward in llms post-training w4a8 quantization using floating-point formats. *arXiv preprint arXiv:2307.09782*, 2023.
- [27] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2:129–146, 2020.
- [28] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *arXiv preprint arXiv:2308.07633*, 2023.
- [29] Mingyang Zhang, Chunhua Shen, Zhen Yang, Linlin Ou, Xinyi Yu, Bohan Zhuang, et al. Pruning meets low-rank

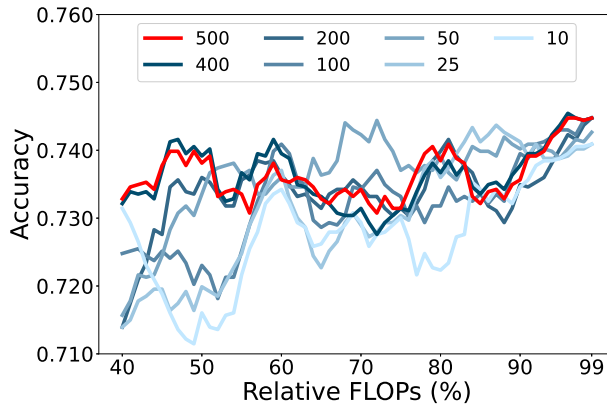


Figure 7: In the MRPC experiment, the performance converges when the number of samples exceeds 400.

- parameter-efficient fine-tuning. *arXiv preprint arXiv:2305.18403*, 2023.
- [30] Azade Nova, Hanjun Dai, and Dale Schuurmans. Gradient-free structured pruning with unlabeled data. *arXiv preprint arXiv:2303.04185*, 2023.
 - [31] Prakhar Ganesh, Yao Chen, Xin Lou, Mohammad Ali Khan, Yin Yang, Hassan Sajjad, Preslav Nakov, Deming Chen, and Marianne Winslett. Compressing large-scale transformer-based models: A case study on bert. *Transactions of the Association for Computational Linguistics*, 9:1061–1080, 2021.
 - [32] Thomas M Cover. *Elements of information theory*. John Wiley & Sons, 1999.
 - [33] Kristoffer Wickstrøm, Sigurd Løkse, Michael Kampffmeyer, Shujian Yu, Jose Principe, and Robert Jenssen. Information plane analysis of deep neural networks via matrix-based renyi’s entropy and tensor kernels. *arXiv preprint arXiv:1909.11396*, 2019.
 - [34] Luis Gonzalo Sanchez Giraldo, Murali Rao, and Jose C Principe. Measures of entropy from data using infinitely divisible kernels. *IEEE Transactions on Information Theory*, 61(1):535–548, 2014.
 - [35] Shujian Yu, Luis Gonzalo Sanchez Giraldo, Robert Jenssen, and Jose C Principe. Multivariate extension of matrix-based renyi’s α -order entropy functional. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(11):2960–2966, 2019.
 - [36] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *arXiv preprint arXiv:1905.09418*, 2019.
 - [37] Zejian Liu, Fanrong Li, Gang Li, and Jian Cheng. Ebert: Efficient bert inference with dynamic structured pruning. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 4814–4823, 2021.
 - [38] Woosuk Kwon, Sehoon Kim, Michael W Mahoney, Joseph Hassoun, Kurt Keutzer, and Amir Gholami. A fast post-training pruning framework for transformers. *Advances in Neural Information Processing Systems*, 35:24101–24116, 2022.
 - [39] Ziqing Yang, Yiming Cui, and Zhigang Chen. TextPruner: A model pruning toolkit for pre-trained language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 35–43, Dublin, Ireland, May 2022. Association for Computational Linguistics.
 - [40] Seungcheol Park, Hojun Choi, and U Kang. Accurate retraining-free pruning for pretrained encoder-based language models. In *The Twelfth International Conference on Learning Representations*, 2023.
 - [41] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
 - [42] François Lagunas, Ella Charlaix, Victor Sanh, and Alexander M Rush. Block pruning for faster transformers. *arXiv preprint arXiv:2109.04838*, 2021.
 - [43] Yongqi An, Xu Zhao, Tao Yu, Ming Tang, and Jinqiao Wang. Fluctuation-based adaptive structured pruning for large language models. *arXiv preprint arXiv:2312.11983*, 2023.
 - [44] Hongfang Zhou, Xiqian Wang, and Rourou Zhu. Feature selection based on mutual information with correlation coefficient. *Applied Intelligence*, pages 1–18, 2022.
 - [45] David D Lewis. Feature selection and feature extraction for text categorization. In *Speech and Natural Language: Proceedings of a Workshop Held at Harriman, New York, February 23-26, 1992*, 1992.
 - [46] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005.
 - [47] Chun Fan, Jiwei Li, Xiang Ao, Fei Wu, Yuxian Meng, and Xiaofei Sun. Layer-wise model pruning based on mutual information. *arXiv preprint arXiv:2108.12594*, 2021.
 - [48] CH Sarvani, Mrinmoy Ghorai, Shiv Ram Dubey, and SH Shabbeer Basha. Hrel: Filter pruning based on high relevance between activation maps and class labels. *Neural Networks*, 147:186–197, 2022.
 - [49] Sungyeop Lee and Junghyo Jo. Information flows of diverse autoencoders. *arXiv preprint arXiv:2102.07402*, 2021.
 - [50] Caroline M Holmes and Ilya Nemenman. Estimation of mutual information for real-valued data with error bars and controlled bias. *Physical Review E*, 100(2):022404, 2019.
 - [51] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical review E*, 69(6):066138, 2004.
 - [52] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. Mutual information neural estimation. In *International Conference on Machine Learning*, pages 531–540. PMLR, 2018.
 - [53] David W Scott. *Multivariate density estimation: theory, practice, and visualization*. John Wiley & Sons, 2015.
 - [54] Trevor F Cox and Michael AA Cox. *Multidimensional scaling*. CRC press, 2000.
 - [55] Nello Cristianini, John Shawe-Taylor, Andre Elisseeff, and Jaz Kandola. On kernel-target alignment. *Advances in Neural Information Processing Systems*, 14, 2001.
 - [56] Samir Khaki and Konstantinos N Plataniotis. The need for speed: Pruning transformers with one recipe. In *The Twelfth International Conference on Learning Representations*, 2024.
 - [57] Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Well-read students learn better: On the importance of pre-training compact models. *arXiv preprint arXiv:1908.08962*, 1908.
 - [58] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
 - [59] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.
 - [60] Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual

- similarity-multilingual and cross-lingual focused evaluation. *arXiv preprint arXiv:1708.00055*, 2017.
- [61] Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Third International Workshop on Paraphrasing (IWP2005)*. Asia Federation of Natural Language Processing, January 2005.
- [62] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.